

Rapport Collectif - SAÉ 2.01

MAUME Timothy - QUEFELEC Neven - TAUZI Elliott - MONLOUIS Guewen

1. Introduction

Ce rapport présente le travail de conception et de développement réalisé dans le cadre de la SAE Brique'UTO. L'objectif principal de ce projet est de concevoir, développer et tester un logiciel en ligne de commande pour un collectionneur passionné de LEGO. Cette application s'appuie sur une base de données relationnelle qui modélise les différentes boîtes, leurs contenus précis, les pièces, les figurines et la hiérarchie des thèmes. Le logiciel créé lors de cette SAE sera la base de la partie 2 de cette SAE lors de laquelle nous créerons une IHM pour ce logiciel. Nous avons réalisé ce projet en collaborant via liveshare principalement.

2. Architecture et Modélisation Orientée Objet

Pour assurer le bon fonctionnement de l'application, nous avons mis en place une architecture logicielle en trois couches distinctes : la couche modèle, la couche d'accès à la base de données, et l'interface utilisateur. Nous avons modélisé les classes fondamentales du projet telles que Theme, Contenu, Piece, Categorie, Couleur et Figurine. Pour gérer la spécificité des boîtes, nous avons implémenté une notion d'héritage où les boîtes composées et les boîtes personnalisées héritent de la classe mère Boîte. Les relations de composition, notamment pour gérer les pièces et les figurines incluses, y sont aussi représentées.

3. Fonctionnalités Implémentées

L'application propose des menus et sous-menus organisés en ligne de commande, et divise ses fonctionnalités selon deux rôles :

- **Pour l'utilisateur (Collectionneur) :**
 - **Exploration et consultation :** L'utilisateur peut rechercher une boîte spécifique par nom ou numéro. Il peut consulter les détails complets d'une boîte, incluant les figurines, les sous-boîtes (pour les packs), ainsi que les pièces (on distingue aussi celles de base et celles fournies en supplément). Des statistiques globales, comme la répartition par couleur et le nombre total de pièces, sont aussi générées.
 - **Recherche avancée :** La méthode rechercherBoitesParTheme(Theme theme) a été implémentée pour récupérer toutes les boîtes appartenant à un thème spécifique, y compris celles de ses sous-thèmes. L'utilisateur peut aussi trouver toutes les boîtes contenant une pièce précise.
 - **Gestion de collection :** Il est possible d'ajouter des boîtes à sa collection, d'en spécifier l'état (complet ou incomplet), et de générer automatiquement la liste des pièces manquantes.
 - **Création sur-mesure :** La méthode composerBoitePersonnalisee(String nom, List<Piece> pieces) permet la création de boîtes virtuelles. L'algorithme vérifie que les pièces existent, calcule le nombre total d'éléments, génère un identifiant unique, et vérifie qu'une combinaison strictement identique n'existe pas déjà dans la base de

données avant d'enregistrer la boîte dans celle-ci.

- **Pour l'administrateur :**

- Il possède les droits pour modifier la base de données en ajoutant de nouvelles boîtes, de nouveaux thèmes ou de nouvelles pièces liées à des catégories. Il peut également mettre à jour le contenu d'une boîte existante.

4. Contraintes techniques et qualité du code

Pour toute la partie développement, nous avons veillé à appliquer les méthodes de conception orientée objet que nous avons vues en cours.

- **Base de données :** Nous avons utilisé la technologie JDBC pour relier notre code Java à la base de données. C'est la toute première partie que nous avons développée afin d'avoir une base avant de faire le reste de l'application.
- **Tests et robustesse :** Pour vérifier que tout fonctionne bien et respecte le sujet, nous avons surtout fait des tests en conditions réelles d'utilisation. Nous avons aussi mis en place une gestion des exceptions un peu partout : de cette façon, si un utilisateur fait une erreur de frappe ou cherche un identifiant qui n'existe pas, le programme gère le problème proprement au lieu de planter brutalement.
- **Gestion des classes :** Nous avons fait attention à bien respecter l'encapsulation dans l'ensemble de nos classes pour garder une structure de code claire. Enfin, nous avons utilisé un dépôt Git tout au long du projet pour garder une trace de notre travail et avoir un historique propre.

5. Conclusion

Ce projet nous a permis de mettre en pratique la chaîne complète de développement d'un logiciel, depuis l'analyse des besoins et l'élaboration de la conception UML, jusqu'à l'implémentation en Java. L'architecture en couches garantit une base solide qui permet à Briqu'IUTO d'être facilement évolutive à plus tard.